# koba::blog

2021-11-19

## リーチ宣言牌のスジは危険？

麻雀 JavaScript

電脳麻将 の牌譜解析機能を npm パッケージ化しました。

- @kobalab/majiang-analog - npm

電脳麻将形式の牌譜を解析する基底クラスを提供します。本クラスのサブクラスを作成し、**解析のためのプログラムを書く** ことができます。天鳳の牌譜も tenhou-log で電脳麻将形式に変換し、解析する ことが可能です。

今回は例題として **リーチ宣言牌のスジが危険か** 解析してみます。いわゆるモロ引っ掛けですね。

### インストール

インストールは以下の方法がありますが、目的に合わせて選択してください。

#### 1. パッケージに同梱されている例が使いたい場合

GitHubに公開している リポジトリ を clone するか、 リリース からダウンロードすればOKです。

clone する場合

```
$ git clone https://github.com/kobalab/majiang-analog.git
```

最新版をダウンロードする場合

```
curl -L https://github.com/kobalab/majiang-analog/archive/master.tar.gz | tar xvfz -
```

npm 対応のパッケージですので、ダウンロード後にリポジトリのルートディレクトリで以下を実行し、関連パッケージをインストールしてください。

```
$ npm ci
```

examples にある例を真似てプログラムを書いてみましょう。README にも説明があります。

#### 2. 独自のパッケージを作る場合

この目的の方はいないと思いますが、npm パッケージの標準的な使い方をすればOKです。

インストールします。

```
$ npm i @kobalab/majiang-analog
```

`require()` の書き方は以下の標準的な方法になります。

```
const analog = require("@kobalab/majiang-analog");
```

## プログラミング

牌譜解析ツールは、プログラム内で実際に牌譜を再生し、適当なタイミングでプログラムのフックを呼び出す形で動作します。フックの種類は README#メソッド を参照してください。牌譜などのデータ構造や手牌などのクラス仕様は以下にドキュメントがあります。

- Home · kobalab/majiang-core Wiki · GitHub

今回は打牌のタイミングで

1. リーチ宣言だった場合
2. 待ちの種類を分類する

というプログラムになりますね。

具体的には打牌に対応するフックを以下のように書きます。

```
    dapai(dapai) {
        let r = this._result;
        if (dapai.p.substr(-1) == '*') {    // リーチがあった場合

            let shoupai = this.board.shoupai[dapai.l];  // リーチ者の手牌を取得

            /* フリテンリーチを除外 */
            let tingpai = Majiang.Util.tingpai(shoupai);
            if (tingpai.find(p=> this.board.he[dapai.l].find(p))) return;

            /* 国士無双のリーチを除外 */
            let mm = Majiang.Util.hule_mianzi(shoupai.clone().zimo(tingpai[0]));
            if (mm[0].length == 13) return;

            let s = dapai.p[0], n = +dapai.p[1]||5;
            if (s == 'z') return;                       // 字牌は集計対象外
            if (n == 4) {                               // 4切りリーチ
                r['4'].n++;
                if (tingpai.find(p=> p == s+'7'))   // 7が待ちにある
                            r['4']['7'][get_tingpai_type(shoupai, s+'7')]++;
            }
            else if (n == 5) {                          // 5切りリーチ
                r['5'].n++;
                if (tingpai.find(p=> p == s+'2'))   // 2が待ちにある
                            r['5']['2'][get_tingpai_type(shoupai, s+'2')]++;
                if (tingpai.find(p=> p == s+'8'))   // 8が待ちにある
                            r['5']['8'][get_tingpai_type(shoupai, s+'8')]++;
            }
            else if (n == 6) {                          // 6切りリーチ
                r['6'].n++;
                if (tingpai.find(p=> p == s+'3'))   // 3が待ちにある
                            r['6']['3'][get_tingpai_type(shoupai, s+'3')]++;
            }
        }
    }
```

完全なプログラムは以下にあります。

- majiang-analog/trap_lizhi.js at master · kobalab/majiang-analog · GitHub

## 解析実行

牌譜のあるディレクトリ[*1]を指定して以下のようにコマンドを実行します。

```
$ node trap_lizhi.js ~/tenhou-log/2019
```

2019年の天鳳鳳凰卓のデータを使いました。

## 解析結果

得られた出力を表にまとめました[*2]。

| | 単騎 | 双碰 | 嵌張 | 辺張 | 両面 | 合計 |
|---|---|---|---|---|---|---|
| スジ28牌 | 0.36% | 0.83% | 2.45% | | | 3.64% |
| スジ37牌 | 0.31% | 0.71% | 3.30% | 1.08% | | 5.40% |

2019年天鳳鳳凰卓での危険度統計 では、以下のようにスジ28牌の危険度は 3.50%、スジ37牌では 4.62% ですので、にわかに **危険になったとは言い難い** ですね。

| | 単騎 | 双碰 | 嵌張 | 辺張 | 両面 | 合計 |
|---|---|---|---|---|---|---|
| スジ28牌 | 0.62% | 1.20% | 1.68% | | | 3.50% |
| スジ37牌 | 0.60% | 0.94% | 1.69% | 1.39% | | 4.62% |

細かくみると、嵌張待ちは増えているものの他の待ち方が減っていることが分かります。モロ引っ掛けの嵌張待ちに引っかかった記憶が強く残るため、誤った印象になるのではないかと推測します。

---

*1: 天鳳の牌譜のダウンロードは tenhou-log でできますが、むやみにダウンロードを試みることは天鳳サーバに負担をかけるので、ここで方法は書きません。解析プログラムを書くスキルのある貴方なら方法はきっと分かるハズです😄

*2: 表形式の集計はプログラム化せず Excel に任せるのが無難です